Machine Intelligence 1

1.3 Multilayer Perceptrons

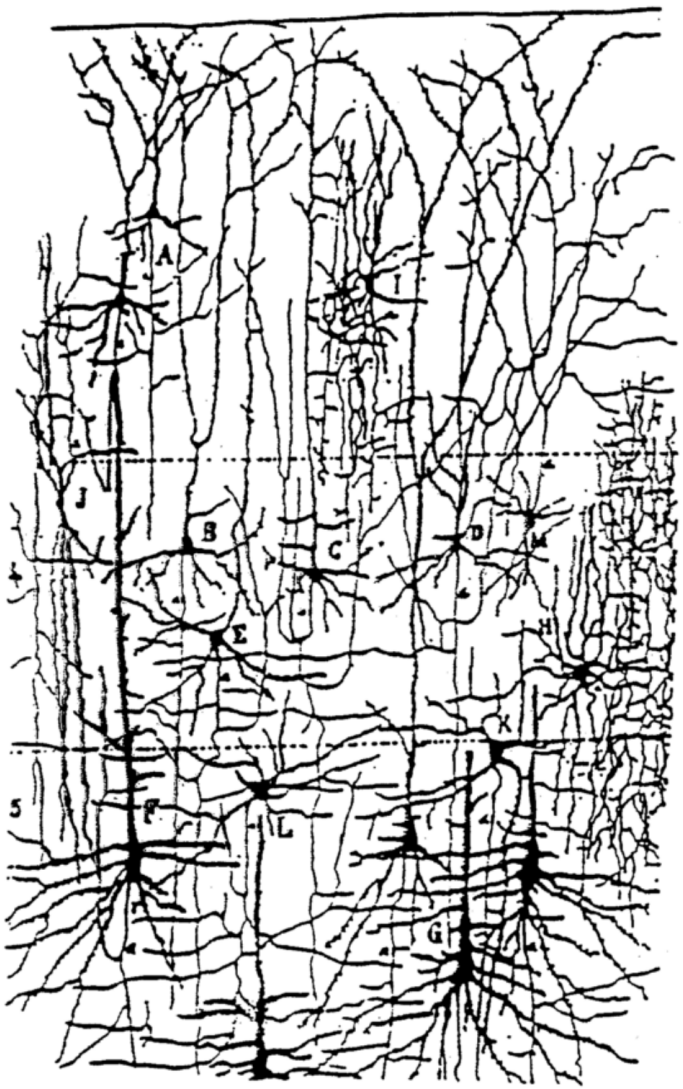
Prof. Dr. Klaus Obermayer

Fachgebiet Neuronale Informationsverarbeitung (NI)

WS 25/26

1.3.1 Classes of Neural Networks

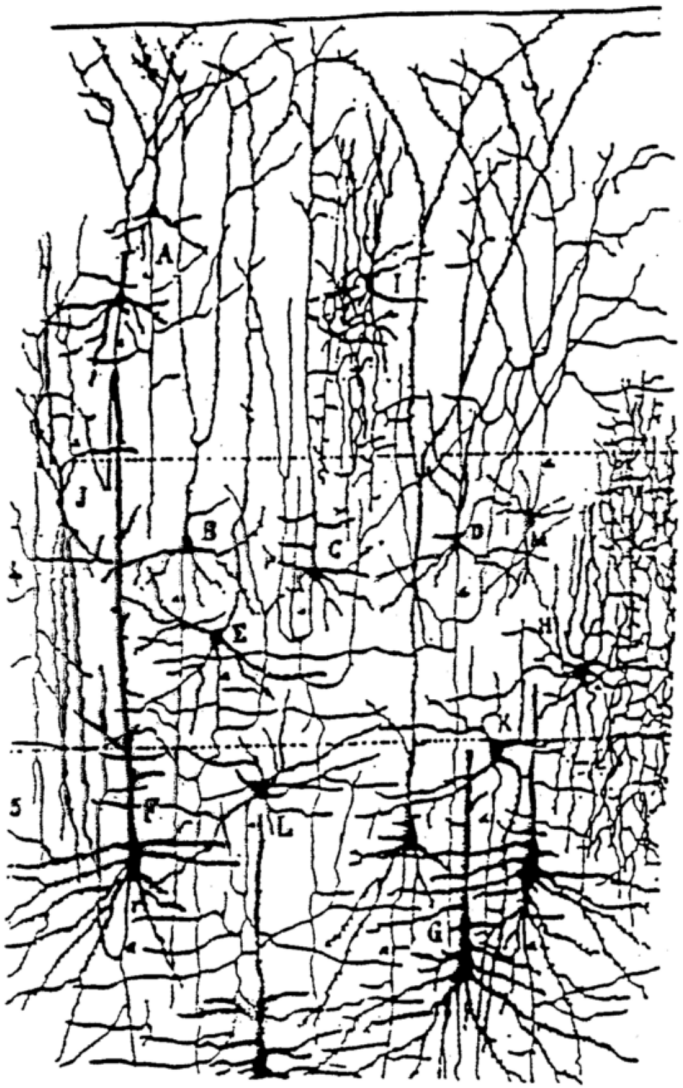
Graphs representing neural networks

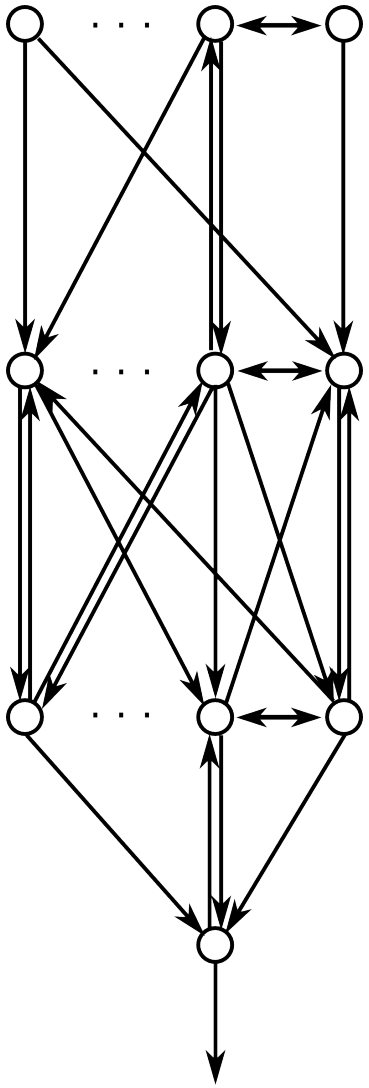


Design principles inspired by biology

- architecture:
  - simple elements
  - massively parallel systems
  - low precision and robustness
  - distributed representation of information
  - no separation between data and program
- inductive learning:
  - data driven, adaptive systems
  - learning and self-organization vs. deduction and programming
  - often seen as a plus: biologically inspired learning rules

Graphs representing neural networks





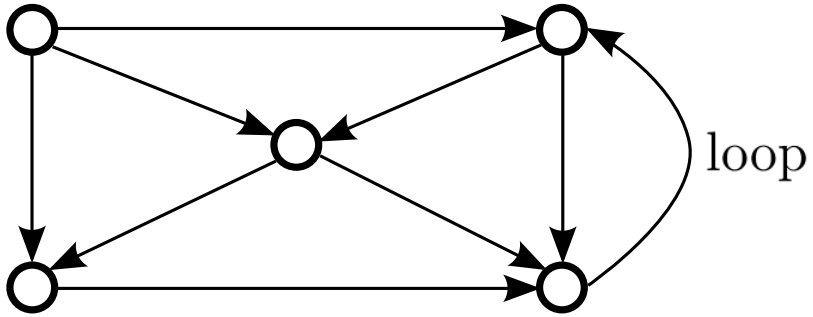
neural network
↓
directed graph

(connectionist) neuron
↓
node of the graph

neural connection
↓
weighted edge

Recurrent Neural Networks (RNNs)

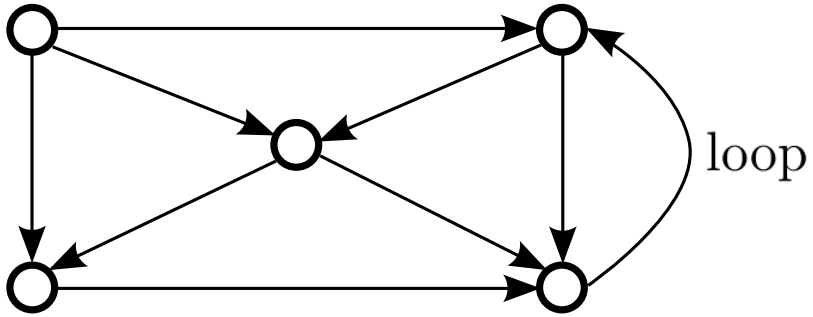
Recurrent networks $\widehat{=}$ directed graphs containing cycles



- dynamical systems
- spatio-temporal pattern analysis
- sequence processing
- associative memory and pattern completion

Recurrent Neural Networks (RNNs)

Recurrent networks $\widehat{=}$ directed graphs containing cycles



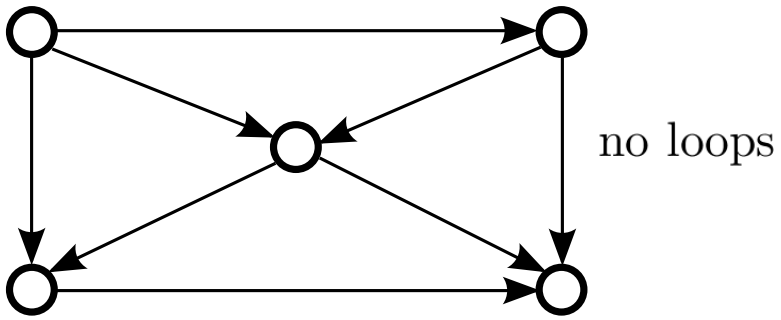
- dynamical systems
- spatio-temporal pattern analysis
- sequence processing
- associative memory and pattern completion

Examples:

Hopfield networks (Hopfield, 1982)
Boltzmann machines (Ackley et al., 1985)
Infinite **I**mpulse **R**esponse (IIR) networks (Crochiere and Oppenheim, 1975)
Long **S**hort-**T**erm **M**emory (LSTM) networks (Hochreiter and Schmidhuber, 1997)
Deep Recurrent Neural Networks (Dauphin et al., 2014)

Feedforward Neural Networks (FFN)

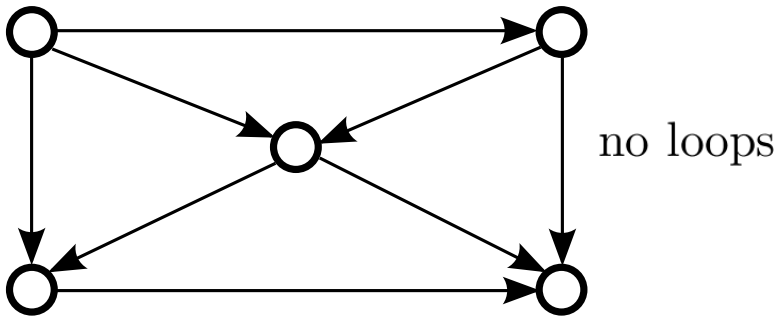
Feedforward Networks $\widehat{=}$ directed acyclic graphs (DAGs)



- association between variables
- prediction of attributes

Feedforward Neural Networks (FFN)

Feedforward Networks $\widehat{=}$ directed acyclic graphs (DAGs)



- association between variables
- prediction of attributes

Examples:

Multil**a**yer-**P**erceptron (MLP, Rumelhart, Hinton & Williams, 1986)
Radial **B**asis **F**unction network (RBF, Broomhead & Lowe, 1988)
Support **V**ector **M**achines (SVM, Cortes & Vapnik, 1995)
Convolutional **N**eural **N**etworks (Fukushima, 1980; Goodfellow et al., 2016)

Prediction of attributes

real-valued attributes: regression problems

$f : \mathbb{R}^N \rightarrow \mathbb{R}$	(or subsets)	one target value
$f : \mathbb{R}^N \rightarrow \mathbb{R}^M$	(or subsets)	multivariate attributes

Prediction of attributes

real-valued attributes: regression problems

$f : \mathbb{R}^N \to \mathbb{R}$	(or subsets)	one target value
$f : \mathbb{R}^N \to \mathbb{R}^M$	(or subsets)	multivariate attributes

ordinal attributes: classification problems

$f : \mathbb{R}^N \to \mathcal{S}$	where $\mathcal{S}$ is a set of attributes $\{a_1, a_2, \ldots, a_M\}$
$f : \mathbb{R}^N \to \{-1, +1\}$	special case: two class problems

Prediction of attributes

real-valued attributes: regression problems

$f : \mathbb{R}^N \to \mathbb{R}$	(or subsets)	one target value
$f : \mathbb{R}^N \to \mathbb{R}^M$	(or subsets)	multivariate attributes

ordinal attributes: classification problems

$f : \mathbb{R}^N \to \mathcal{S}$	where $\mathcal{S}$ is a set of attributes $\{a_1, a_2, \ldots, a_M\}$
$f : \mathbb{R}^N \to \{-1, +1\}$	special case: two class problems

predicting probabilities

$f_k : \mathbb{R}^N \to [0, 1] \subset \mathbb{R}$	probability of attribute a_k from $\mathcal{S}$
s.t. $\sum_{k=1}^{M} f_k(\cdot) = 1$	constrains all $f_k(\cdot)$ to be probabilities

Prediction of attributes

real-valued attributes: regression problems

$f : \mathbb{R}^N \to \mathbb{R}$	(or subsets)	one target value
$f : \mathbb{R}^N \to \mathbb{R}^M$	(or subsets)	multivariate attributes

ordinal attributes: classification problems

$f : \mathbb{R}^N \to \mathcal{S}$	where $\mathcal{S}$ is a set of attributes $\{a_1, a_2, \ldots, a_M\}$
$f : \mathbb{R}^N \to \{-1, +1\}$	special case: two class problems

predicting probabilities

$f_k : \mathbb{R}^N \to [0, 1] \subset \mathbb{R}$	probability of attribute a_k from $\mathcal{S}$
s.t. $\sum_{k=1}^{M} f_k(\cdot) = 1$	constrains all $f_k(\cdot)$ to be probabilities

structured input / structured output

$f : \mathcal{X} \to \mathcal{Y}$ where $\mathcal{X}$ and $\mathcal{Y}$ are sets of structures, e.g. graphs, sentences, or "tensors"

1.3.2 The Multilayer-Perceptron for Regression

Overview inductive learning

data representation

↓

model class

↓

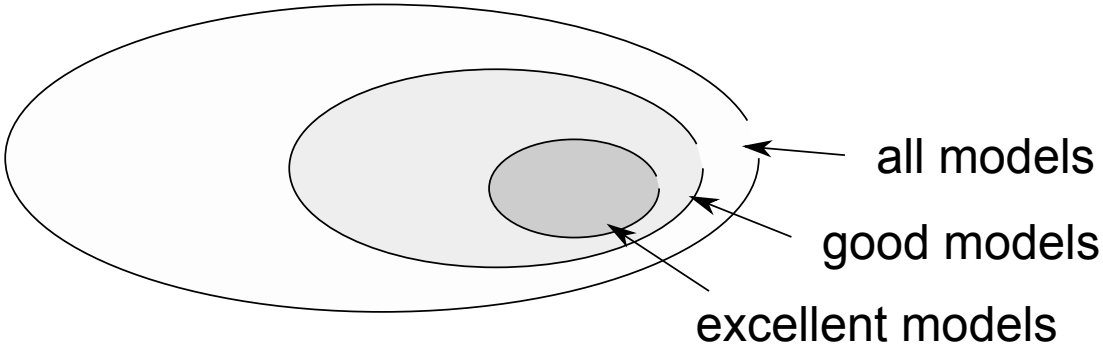
performance measure

↓

optimization

↓

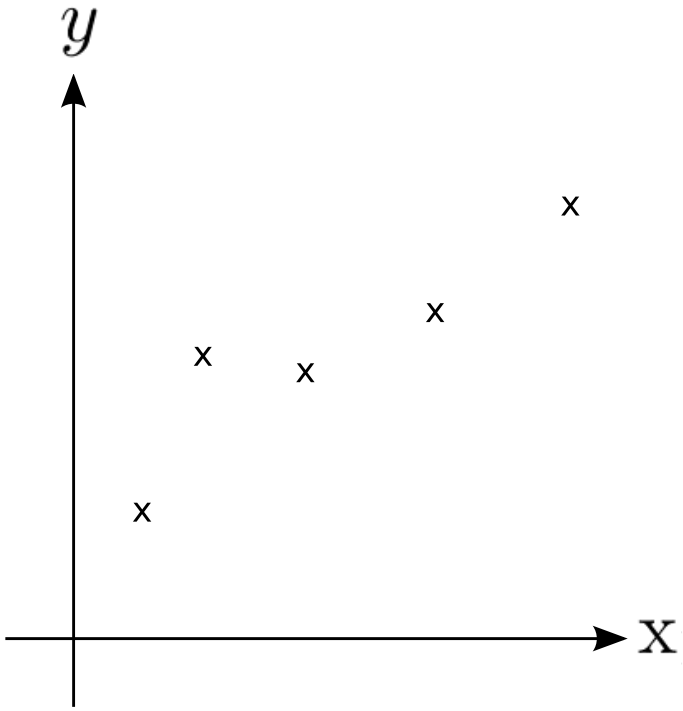
validation



Data Representation

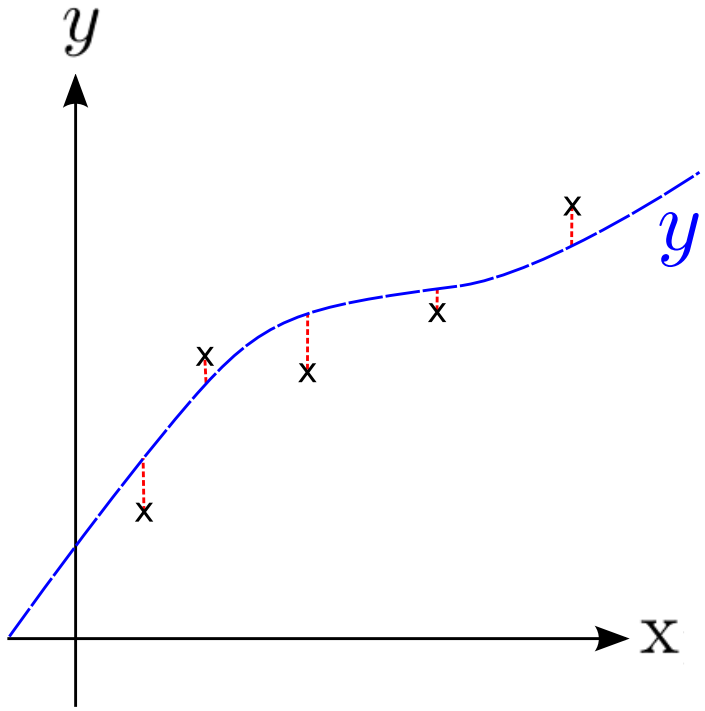
Our example: regression with one real-valued attribute

- input feature vectors: $\underline{\mathbf{x}} \in \mathbb{R}^N$
- output attributes: $y_T \in \mathbb{R}$
- training set: $\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\}, \alpha = 1, \ldots, p$



Our example: regression with one real-valued attribute

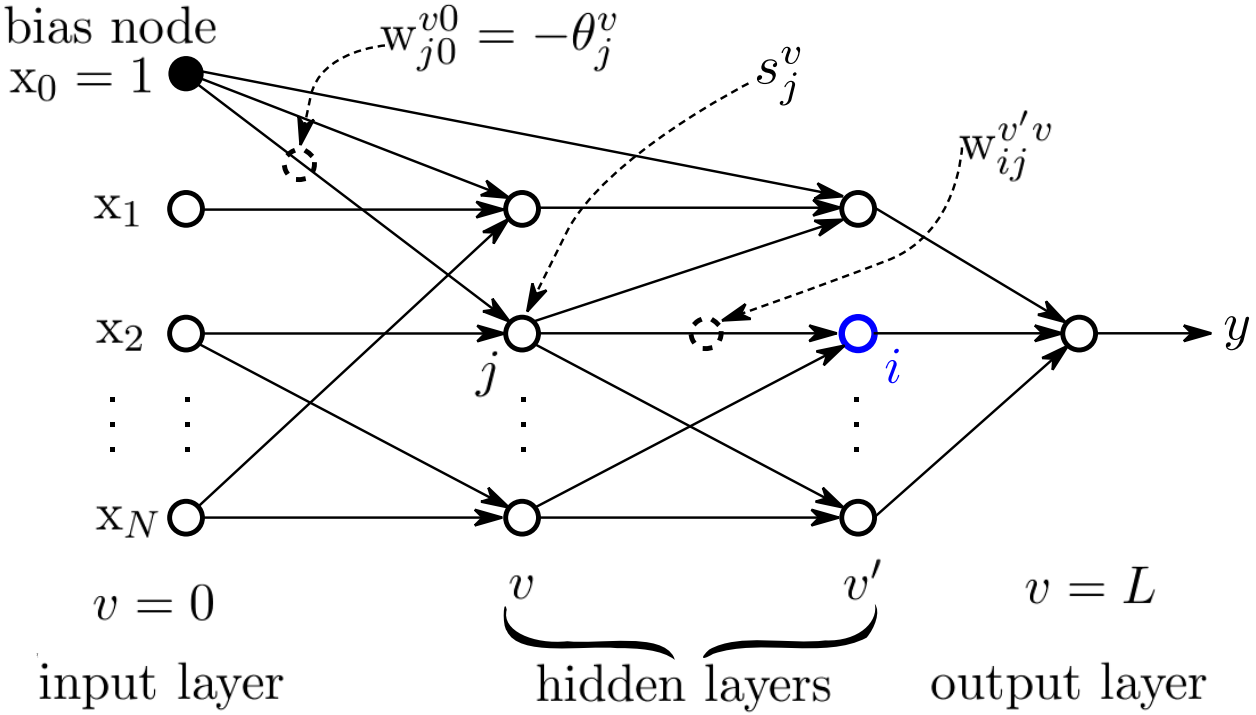
- input feature vectors: $\underline{\mathbf{x}} \in \mathbb{R}^N$
- output attributes: $y_T \in \mathbb{R}$
- training set: $\{\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\}, \alpha = 1, \ldots, p$
- label function: $y_T = y(\underline{\mathbf{x}}) + \text{noise}$



Model Class

Multilayer perceptrons (MLPs)

- layered FFN $y(\cdot;\underline{\mathbf{w}})$ models label function $y_T(\cdot)$

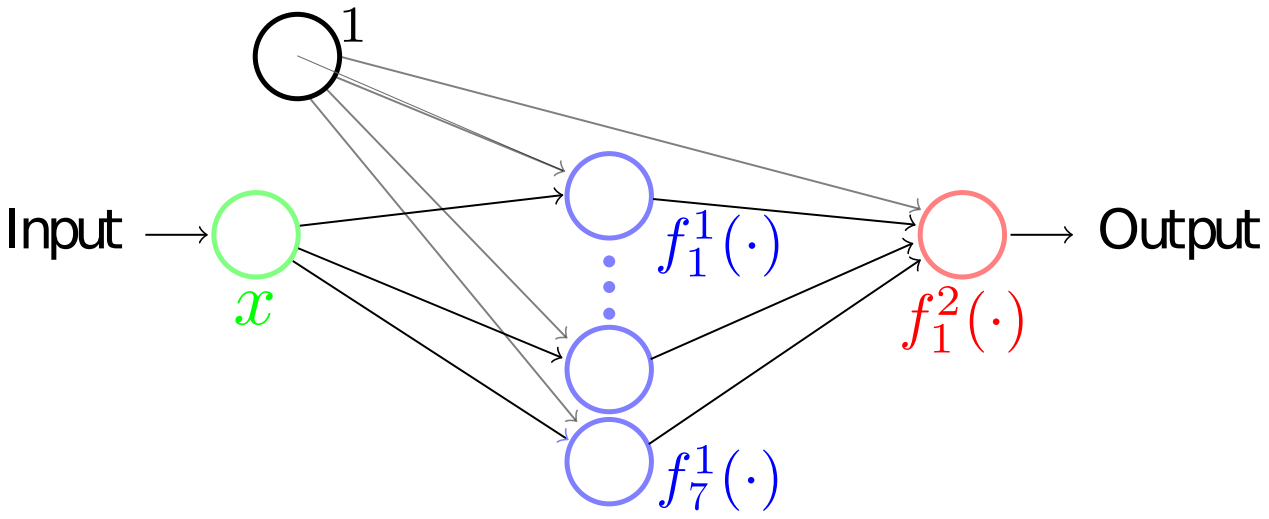


$$s_i^{v'} = f_i^{v'}(h_i^{v'}) \overset{\text{e.g.}}{=} \tanh(h_i^{v'}), \qquad h_i^{v'} = \sum_{j=0}^{N_v} w_{ij}^{v'v} s_j^v$$

$$y = f_1^L(h_1^L) \overset{\text{e.g.}}{=} h_1^L = \sum_{j=0}^{N_{v'}} w_{1j}^{Lv'} s_j^{v'}$$

Model class: one example

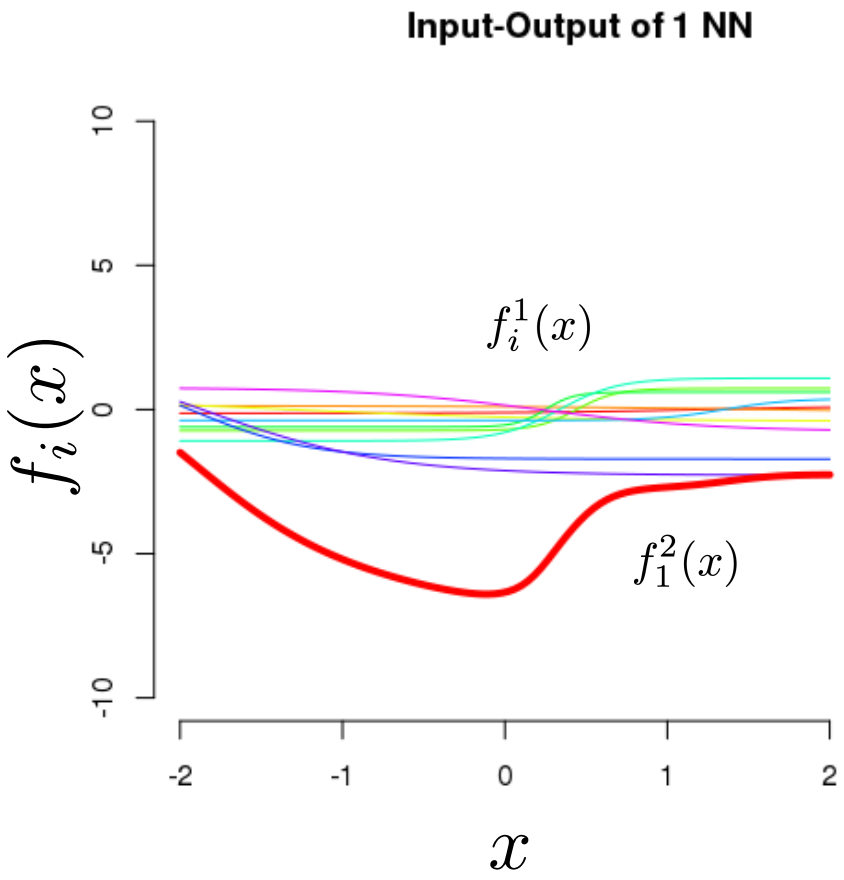
1 input, 1 output, 1 hidden layer with 7 units

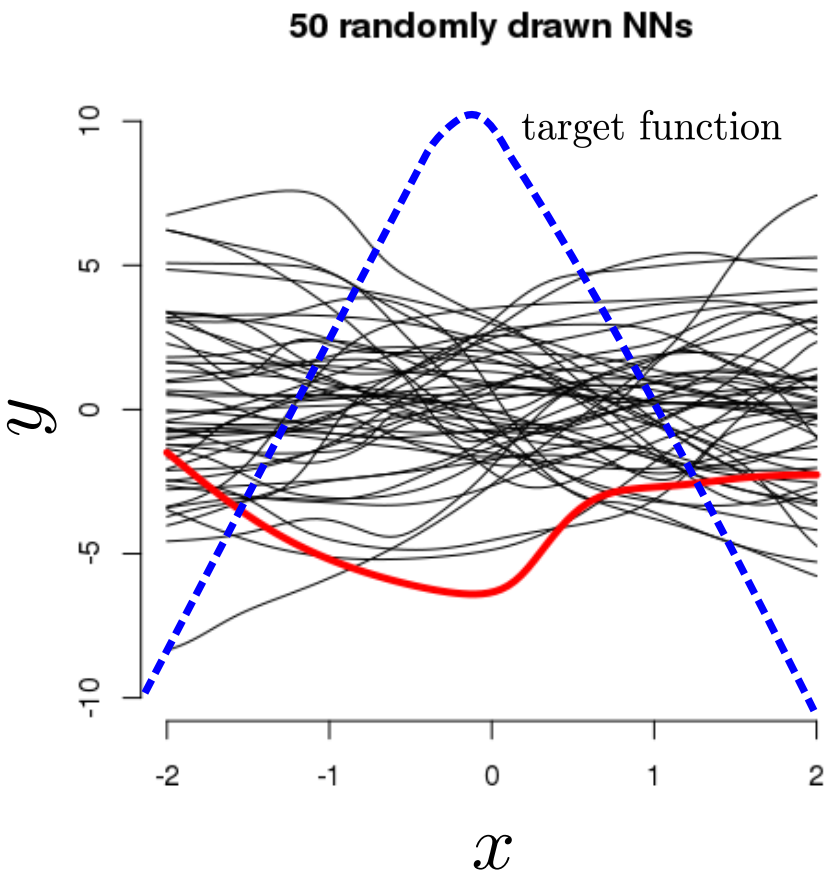


$$y(x; \underline{\mathbf{W}}^{10}, \underline{\mathbf{w}}^{21}) \quad = \quad f_1^2\Big(\sum_{i=1}^{7} w_{1\,i}^{21}\; f_i^1(w_{i1}^{10}\, x - w_{i0}^{10}) - w_{10}^{20}\Big)$$

$$f_i^1(a) = \tanh(a) \;\;\&\;\; f_1^2(a) = a$$

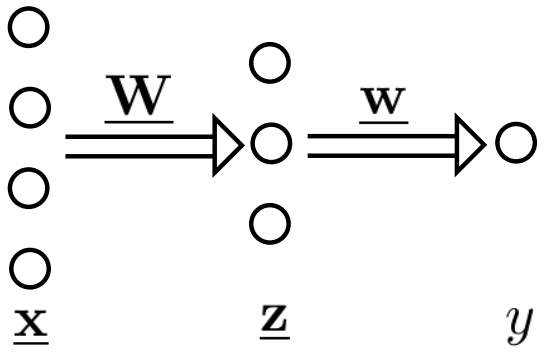
Model class: one example





(parameters drawn from Gaussian distribution)

Linear transfer functions



$$
\begin{aligned}
y &= \underline{\mathbf{w}}^\top \underline{\mathbf{z}} = \underline{\mathbf{w}}^\top \underline{\mathbf{W}}\,\underline{\mathbf{x}} \\
&= (\underline{\mathbf{W}}^\top \underline{\mathbf{w}})^\top \underline{\mathbf{x}} = \underline{\widehat{\mathbf{w}}}^\top \underline{\mathbf{x}} \\
&\widehat{=} \text{ connectionist neuron}
\end{aligned}
$$

MLPs are universal approximators

Funahashi (1989)

Let $y_T(\underline{\mathbf{x}})$ be a continuous real valued function on a compact subset of $\mathbb{R}^N$ and

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = \sum_{j=1}^{M} w_{1j}^{21} f^1\Big(\sum_{i=1}^{N} w_{ji}^{10} x_i - w_{j0}^{10}\Big) - w_{10}^{20}$$

be a three-layered MLP with a non-constant, bounded, monotonously increasing, and continuous function $f^1 : \mathbb{R} \to \mathbb{R}$.

MLPs are universal approximators

Funahashi (1989)

Let $y_T(\underline{\mathbf{x}})$ be a continuous real valued function on a compact subset of $\mathbb{R}^N$ and

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = \sum_{j=1}^{M} w_{1j}^{21} f^1\Big(\sum_{i=1}^{N} w_{ji}^{10} x_i - w_{j0}^{10}\Big) - w_{10}^{20}$$

be a three-layered MLP with a non-constant, bounded, monotonously increasing, and continuous function $f^1 : \mathbb{R} \to \mathbb{R}$.

Then there exists a set of parameters $M \in \mathbb{N}$ and $\mathrm{w}_{1j}^{21}, \mathrm{w}_{ji}^{10} \in \mathbb{R}$ for every $\varepsilon > 0$ such that

$$\max_{\underline{\mathbf{x}} \in K} \Big| y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) \Big| \leq \varepsilon.$$

(Funahashi, 1989; Hornik et al., 1989)

1.3.3 Performance Measures and Model Selection

Cost functions

$$\underbrace{\underline{\mathbf{x}} \in \mathbb{R}^N}_{\text{feature vector}} \longrightarrow \underbrace{y \in \mathbb{R}}_{\text{attribute}}$$

y_T: true value of attribute

$y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$: predicted value of attribute (e.g. by an MLP with weights $\underline{\mathbf{w}}$)

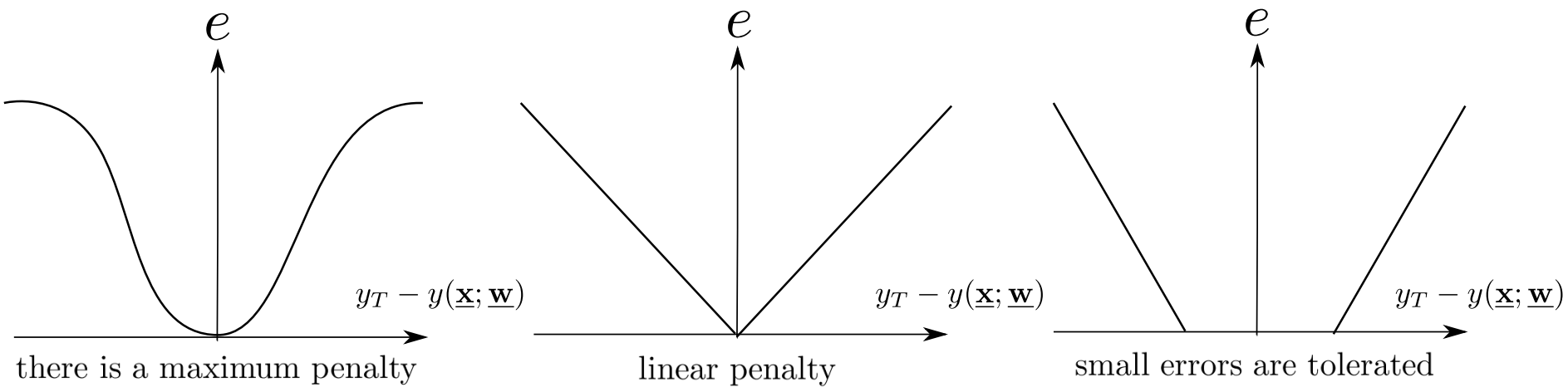
Cost functions

$$\underbrace{\underline{\mathbf{x}} \in \mathbb{R}^N}_{\text{feature vector}} \longrightarrow \underbrace{y \in \mathbb{R}}_{\text{attribute}}$$

y_T: true value of attribute

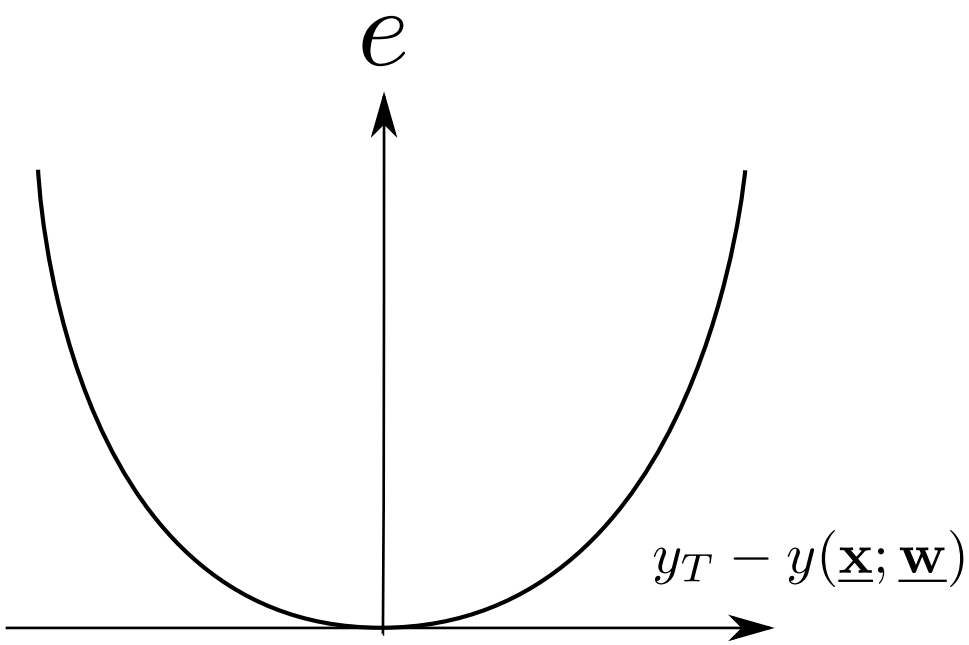
$y(\underline{\mathbf{x}}; \underline{\mathbf{w}})$: predicted value of attribute (e.g. by an MLP with weights $\underline{\mathbf{w}}$)

individual cost $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$



several choices $\Rightarrow$ the optimal predictor will depend on the cost function

Lecture example: quadratic error



$$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) \quad = \quad \frac{1}{2}\Big(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\Big)^2$$

- Gaussian noise on attributes
- sensitive against "outliers"

Performance measure

Generalization error

$$E^G_{[\underline{\mathbf{w}}]} \quad := \quad \langle\, e \,\rangle_{y_T, \underline{\mathbf{x}}} \quad = \quad \iint d\underline{\mathbf{x}}\, dy_T \; P(y_T, \underline{\mathbf{x}})\, e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$$

$P(y_T, \underline{\mathbf{x}})$: joint Probability Density Function (PDF) of observations

Performance measure

Generalization error

$$E^G_{[\underline{\mathbf{w}}]} \quad := \quad \langle\, e \,\rangle_{y_T, \underline{\mathbf{x}}} \quad = \quad \iint d\underline{\mathbf{x}}\, dy_T \; P(y_T, \underline{\mathbf{x}})\, e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)$$

$P(y_T, \underline{\mathbf{x}})$: joint Probability Density Function (PDF) of observations

"good" predictor: low value of E^G
"bad" predictor: high value of E^G

$$E^G_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

but: $P(y_T, \underline{\mathbf{x}})$ is not known

Principle of Empirical Risk Minimization (ERM)

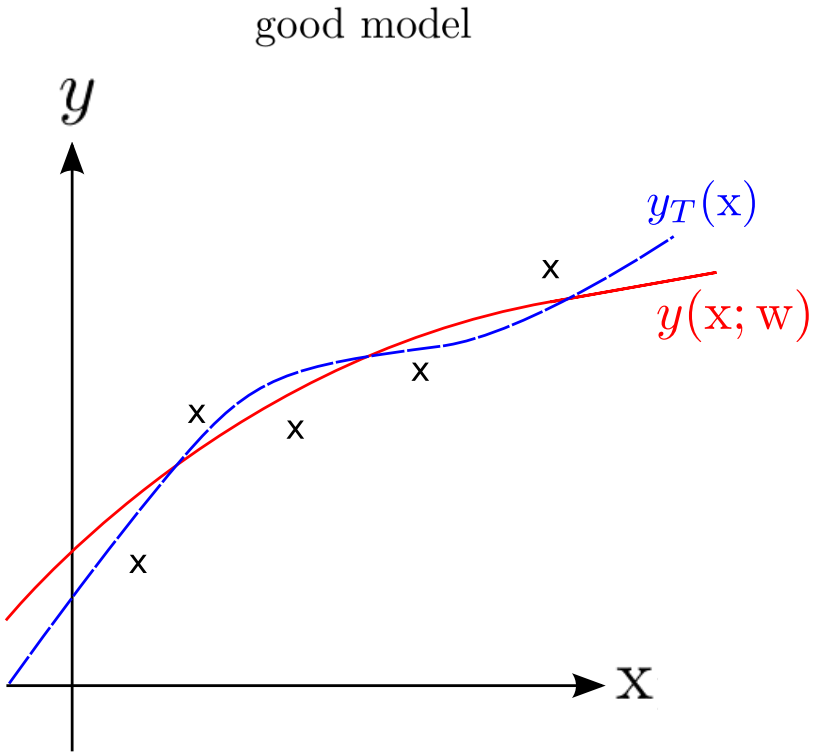
math. expectation (E^G) $\rightarrow$ empirical average (E^T)

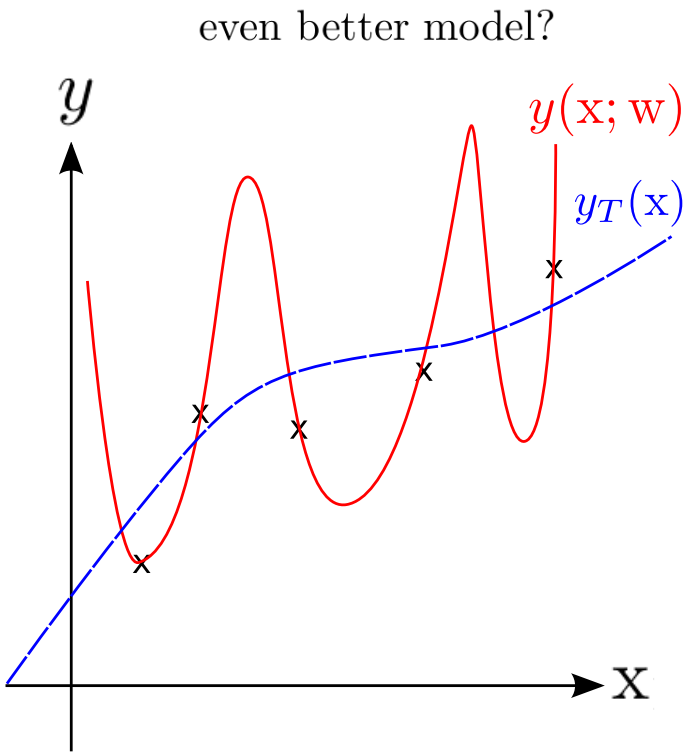
$$E^G_{[\underline{\mathbf{w}}]} := \iint d\underline{\mathbf{x}}\, dy_T P(y_T, \underline{\mathbf{x}})\, e(y_T, y(\underline{\mathbf{x}}; \underline{\mathbf{w}})) \;\rightarrow\; E^T_{[\underline{\mathbf{w}}]} := \frac{1}{p} \sum_{\alpha=1}^{p} e(y_T^{(\alpha)}, y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}))$$

generalization error $\rightarrow$ training error

$$E^G_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \;\rightarrow\; E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

Consequences?



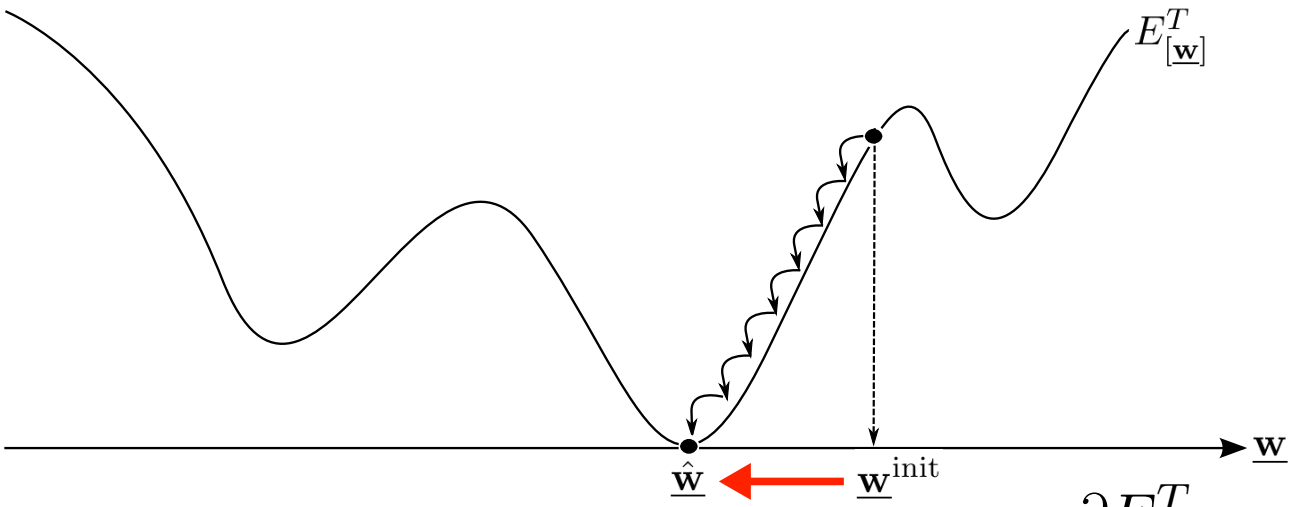


1.3.4 Optimization of Model Parameters: Gradient Descent

Gradient descent

Training error E^T for the given training set:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\big(y_T^{(\alpha)}, y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\big)}_{=:e^{(\alpha)}}$$



$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \quad - \quad \underbrace{\eta}_{\text{learning step}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \mathrm{w}_{ij}^{v'v}}}_{\text{component in gradient vector}}$$

Calculation of the gradient

$$
\begin{aligned}
\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial \mathrm{w}^{v'v}_{ij}} &= \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial \overbrace{e\big(y_T^{(\alpha)}, y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\big)}^{\text{individual cost } e^{(\alpha)}}}{\partial \mathrm{w}^{v'v}_{ij}} \\
&= \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\big(y_T^{(\alpha)}, y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\big)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}}_{\text{factor depending on the cost function}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial \mathrm{w}^{v'v}_{ij}}}_{\text{factor depending on the model class (e.g. MLP)}}
\end{aligned}
$$

Calculation of the error term

Quadratic error

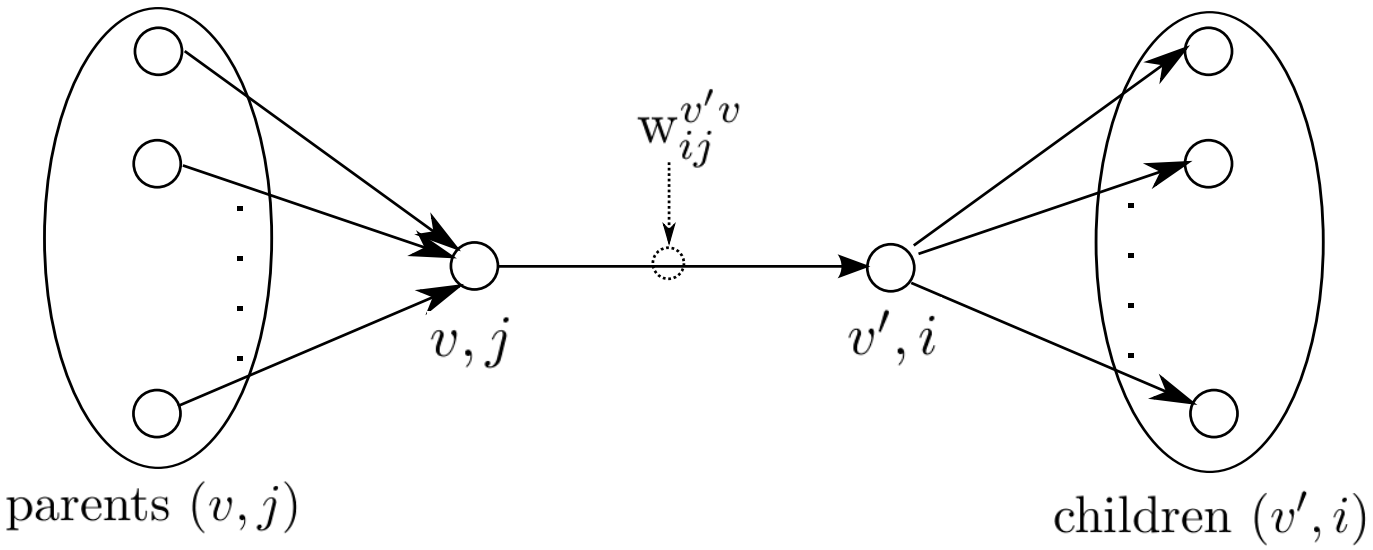
$$e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2$$

$$\Rightarrow \frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)} = y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right) - y_T$$

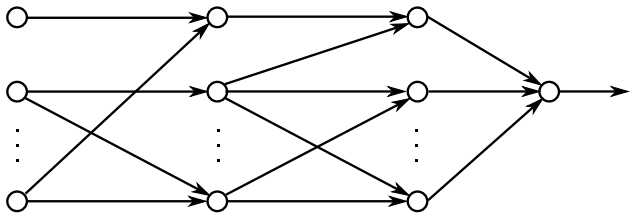
1.3.5 The Backpropagation Method

Gradients in neural networks

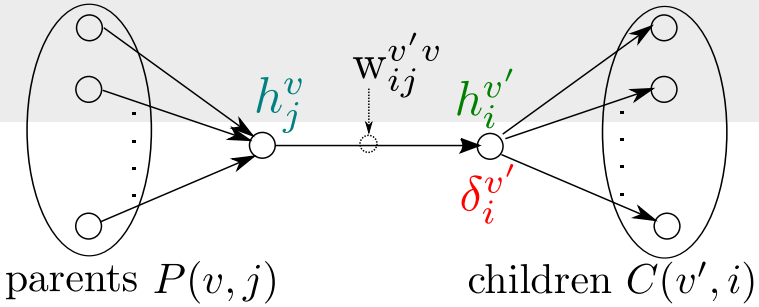
$$\frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial \mathrm{w}_{ij}^{v'v}}$$



Bias node included in the parents.



Gradients in neural networks

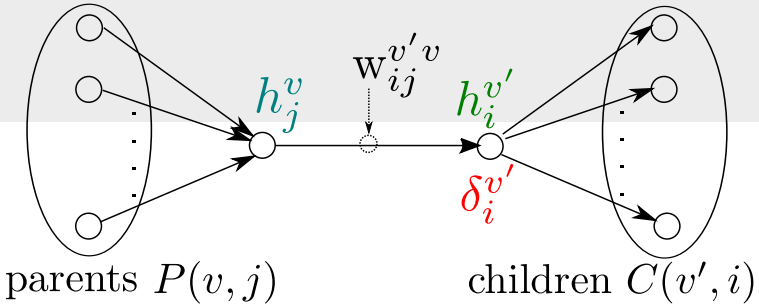


Solution: smart application of the chain rule

$$h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}(h_k^{\mu})$$

$$\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial \mathrm{w}_{ij}^{v'v}} = \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_i^{v'}}}_{\substack{:= \delta_i^{v'} \\ \text{“local error”} \\ \text{at neuron} \\ (v',i)}} \cdot \underbrace{\frac{\partial h_i^{v'}}{\partial \mathrm{w}_{ij}^{v'v}}}_{\substack{= f_j^v(h_j^v) \\ \text{activity} \\ \text{of neuron} \\ (v,j)}}$$

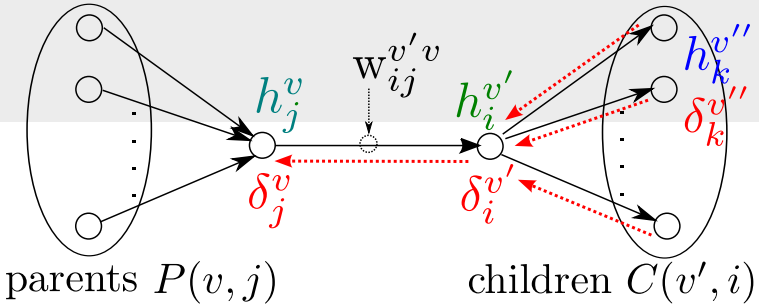
Gradients in neural networks



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

$$h_j^0 := x_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu(h_k^\mu)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

Gradients in neural networks



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

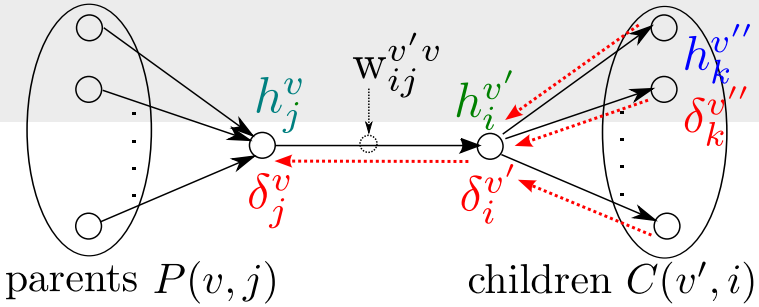
$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu(h_k^\mu)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\rightarrow$ parents)

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{\mathrm{w}_{ki}^{v''v'}\cdot\,[f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''}\,\mathrm{w}_{ki}^{v''v'}$$

Gradients in neural networks



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu(h_k^\mu)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\rightarrow$ parents)

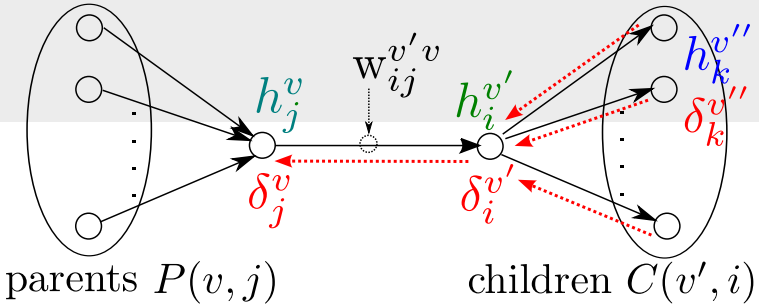
$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{\mathrm{w}_{ki}^{v''v'}\cdot [f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''}\, \mathrm{w}_{ki}^{v''v'}$$

3. **gradient of the individual cost**: using activities and local errors

$$\frac{\partial e(y_T, y(\underline{\mathbf{x}};\underline{\mathbf{w}}))}{\partial \mathrm{w}_{ij}^{v'v}} = \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial \mathrm{w}_{ij}^{v'v}}$$

Gradients in neural networks



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu(h_k^\mu)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\rightarrow$ parents)

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{\mathrm{w}_{ki}^{v''v'}\cdot [f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''}\, \mathrm{w}_{ki}^{v''v'}$$

3. **gradient of the individual cost**: using activities and local errors

$$\frac{\partial e(y_T, y(\underline{\mathbf{x}};\underline{\mathbf{w}}))}{\partial \mathrm{w}_{ij}^{v'v}} = \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \delta_i^{v'} \cdot f_j^v(h_j^v)$$

Summary of backpropagation for gradient descent

initialization of weights and thresholds

while *stopping criterion not met* **do**

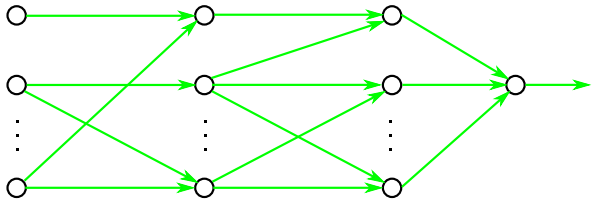
$\quad$ gradient$_{ij}^{v'v} := 0\,, \quad \forall w_{ij}^{v'v}$

$\quad$ **for** $\alpha \in \{1,\ldots,p\}$ **do**

$\quad\quad h_i^0 := x_i^{(\alpha)}\,, \quad \forall i$ // forward propagation

$\quad\quad$ **for** $v' \in \{1,\ldots,L\}$ **do**

$$\quad\quad\quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \underbrace{f_k^\mu\left(h_k^\mu\right)}_{x_k^{(\alpha)} \text{ if } v'=1}\,, \quad \forall i$$

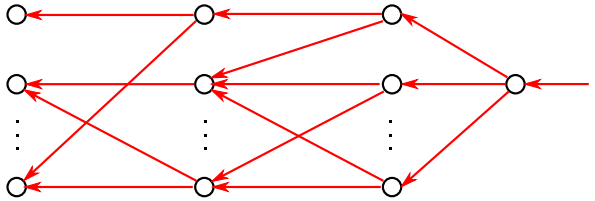


$\quad\quad$ **end**

$\quad\quad \delta_1^L := [f_1^L]'(h_1^L)$ // backward propagation

$\quad\quad$ **for** $v' \in \{L-1,\ldots,1\}$ **do**

$$\quad\quad\quad \delta_i^{v'} := [f_i^{v'}]'(h_i^{v'}) \sum_{(\mu,k)\in C(v',i)} \delta_k^\mu\, w_{ki}^{\mu v'}\,, \quad \forall i$$



$\quad\quad$ **end**

$\quad\quad$ gradient$_{ij}^{v'v}$:= gradient$_{ij}^{v'v} + \frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})}\, \delta_i^{v'}\, f_j^v(h_j^v)\,, \quad \forall w_{ij}^{v'v}$ // sum

$\quad$ **end**

$\quad$ gradient$_{ij}^{v'v}$:= $\frac{1}{p}$gradient$_{ij}^{v'v}$

$\quad w_{ij}^{v'v} := w_{ij}^{v'v} - \eta\,$gradient$_{ij}^{v'v}\,, \quad \forall w_{ij}^{v'v}$ // gradient descent step

end

Summary of backpropagation for gradient descent

initialization of weights and thresholds

while *stopping criterion not met* **do**

$\quad$ gradient$_{ij}^{v'v} := 0\,, \quad \forall w_{ij}^{v'v}$

$\quad$ **for** $\alpha \in \{1,\ldots,p\}$ **do**

$\quad\quad h_i^0 := x_i^{(\alpha)}\,, \quad \forall i$ // forward propagation

$\quad\quad$ **for** $v' \in \{1,\ldots,L\}$ **do**

$$\quad\quad\quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \underbrace{f_k^\mu\left(h_k^\mu\right)}_{x_k^{(\alpha)} \text{ if } v'=1}\,, \quad \forall i$$

$\quad\quad$ **end**

$\quad\quad \delta_1^L := [f_1^L]'(h_1^L)$ // backward propagation

$\quad\quad$ **for** $v' \in \{L-1,\ldots,1\}$ **do**

$$\quad\quad\quad \delta_i^{v'} := [f_i^{v'}]'(h_i^{v'}) \sum_{(\mu,k)\in C(v',i)} \delta_k^\mu\, w_{ki}^{\mu v'}\,, \quad \forall i$$

$\quad\quad$ **end**

$$\quad\quad \text{gradient}_{ij}^{v'v} := \text{gradient}_{ij}^{v'v} + \frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)};\underline{\mathbf{w}})}\, \delta_i^{v'}\, f_j^v(h_j^v)\,, \quad \forall w_{ij}^{v'v} \quad \text{// sum}$$

$\quad$ **end**

$\quad$ gradient$_{ij}^{v'v} := \frac{1}{p}$gradient$_{ij}^{v'v}$

$\quad w_{ij}^{v'v} := w_{ij}^{v'v} - \eta\,\text{gradient}_{ij}^{v'v}\,, \quad \forall w_{ij}^{v'v}$ // gradient descent step

end

computational and memory complexity

$\mathcal{O}(n)$,

n: number of weights

Initialization

- Avoid exponentially reducing/magnifying activities and gradients:

$$var(s^{\nu+1}) = var(s^{\nu}) \qquad var\left(\frac{\partial y}{\partial h^{\nu}}\right) = var\left(\frac{\partial y}{\partial h^{\nu+1}}\right)$$

- Heuristic for squashing functions, e.g. logistic function, $\tanh$ (Glorot and Bengio (2010)):

$$w_{ij}^{\nu'\nu} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_\nu + n_{\nu'}}}, \frac{\sqrt{6}}{\sqrt{n_\nu + n_{\nu'}}}\right]$$

- Heuristic for rectified linear functions (He et al. (2015)):

$$w_{ij}^{\nu'\nu} \sim \mathcal{N}\left(0, \frac{2}{n_\nu}\right)$$

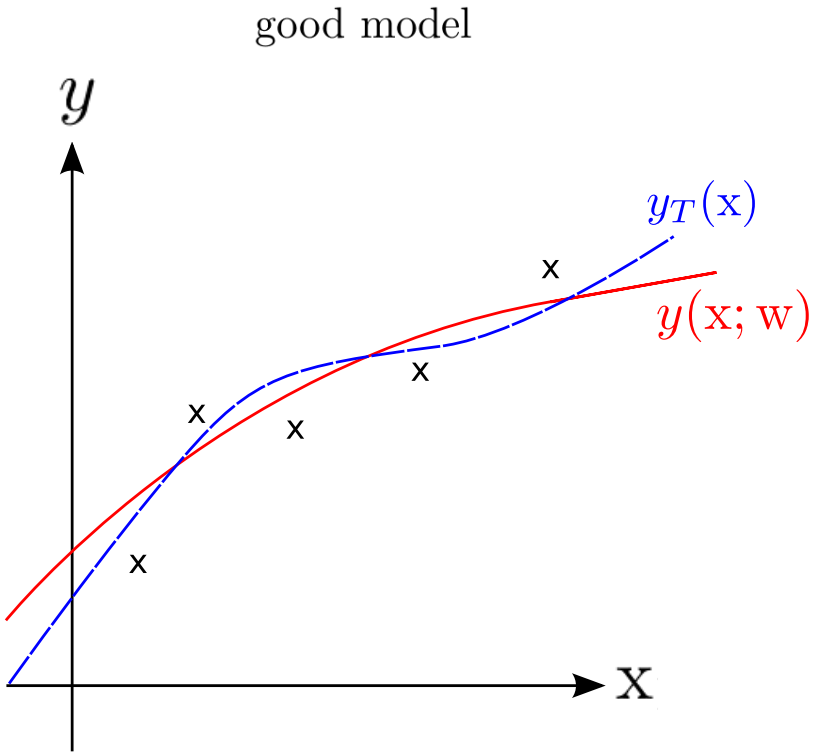
n_ν: number of neurons in layer ν
s^ν, h^ν: indicate the values irrespective of the node index within the layer ν
$\mathcal{U}$: uniform distribution

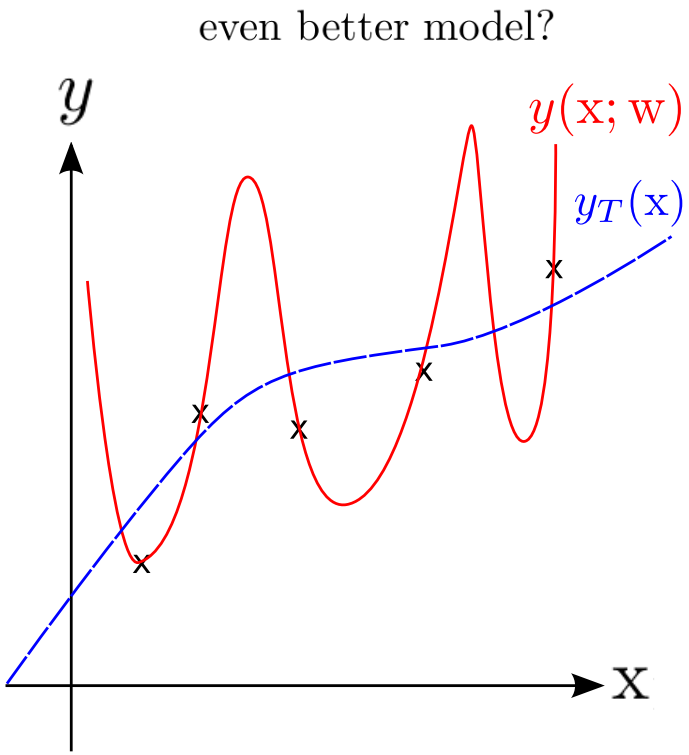
Stopping criteria

- $\rightsquigarrow$ fixed number of iterations
- $\rightsquigarrow$ fixed CPU-time
- $\rightsquigarrow$ E^T falls below a predefined value
- $\rightsquigarrow$ $\frac{\Delta E^T}{E^T}$ falls below a predefined value
- $\rightsquigarrow$ validation criterion fulfilled

1.3.6 Validation of Model Selection

Overfitting





Assessment of prediction quality

Test set method

observations $\begin{cases} \text{training data } \left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}, \quad \alpha \in \{1, \ldots, p\} \\ \rightarrow E^T \text{ is used to select model parameters} \end{cases}$

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} e\big(y_T^{(\alpha)}, y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\big)$$

Assessment of prediction quality

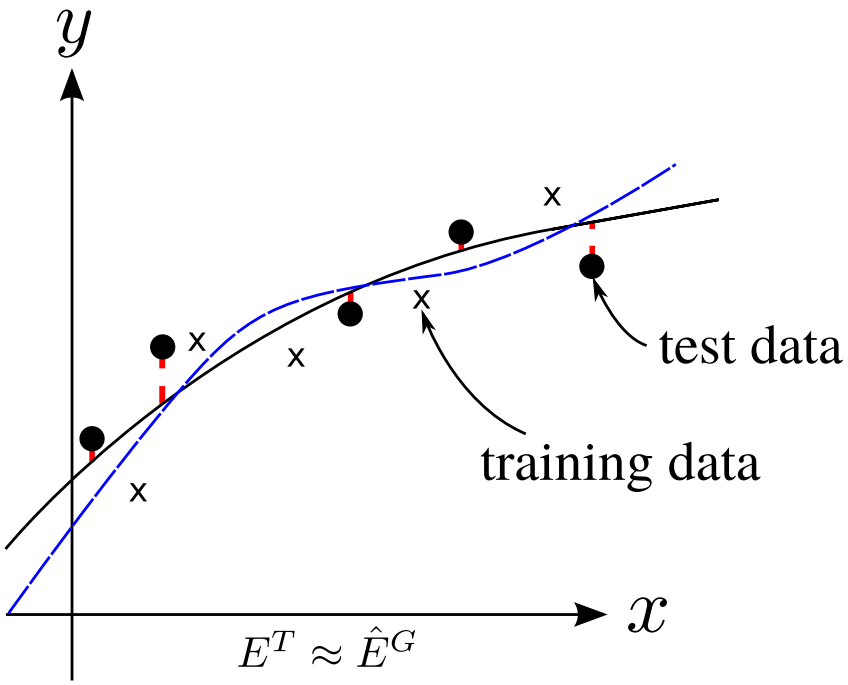
Test set method

$$\text{observations} \begin{cases} \text{training data } \left\{\left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)}\right)\right\}, \quad \alpha \in \{1,\ldots,p\} \\ \rightarrow E^T \text{ is used to select model parameters} \\ \text{test data } \left\{\left(\underline{\mathbf{x}}^{(\beta)}, y_T^{(\beta)}\right)\right\}, \quad \beta \in \{1,\ldots,q\} \\ \rightarrow \hat{E}^G \text{ estimates generalization error } E^G \end{cases}$$

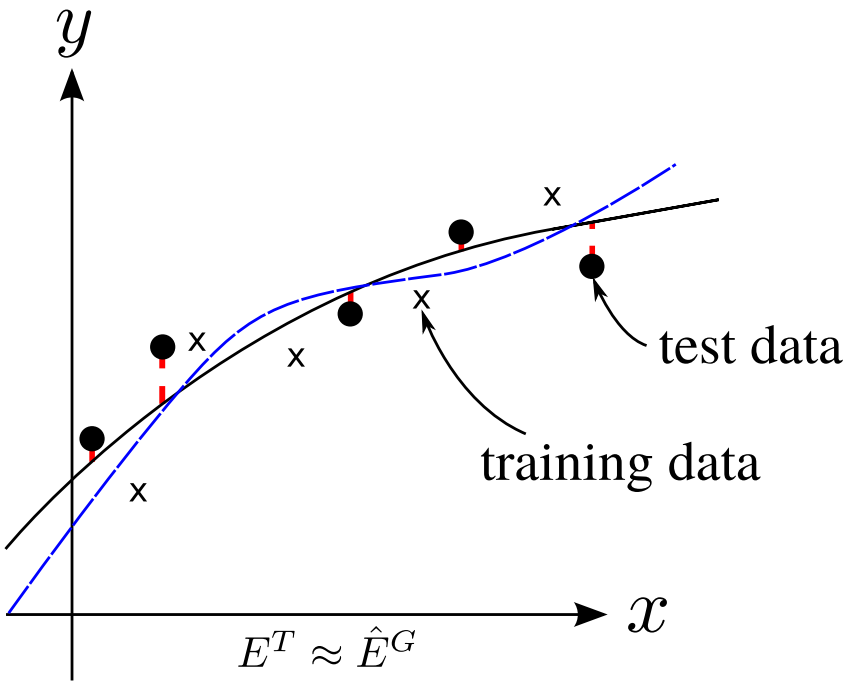
$$\hat{E}^G_{[\underline{\mathbf{w}}]} \quad = \quad \frac{1}{q}\sum_{\beta=1}^{q} e\big(y_T^{(\beta)}, y(\underline{\mathbf{x}}^{(\beta)}; \underline{\mathbf{w}})\big)$$

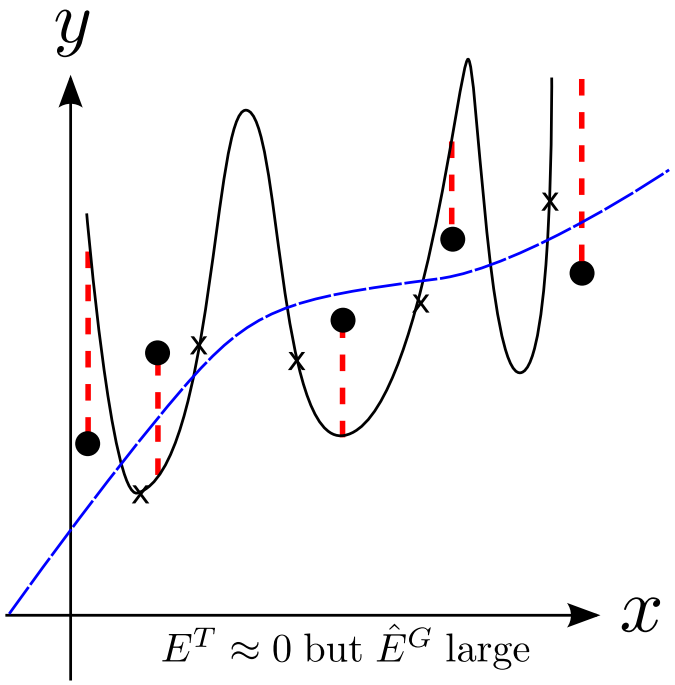
$\hat{E}^G_{[w]}$ is an unbiased estimator of the generalization error of the trained model.

Test set method



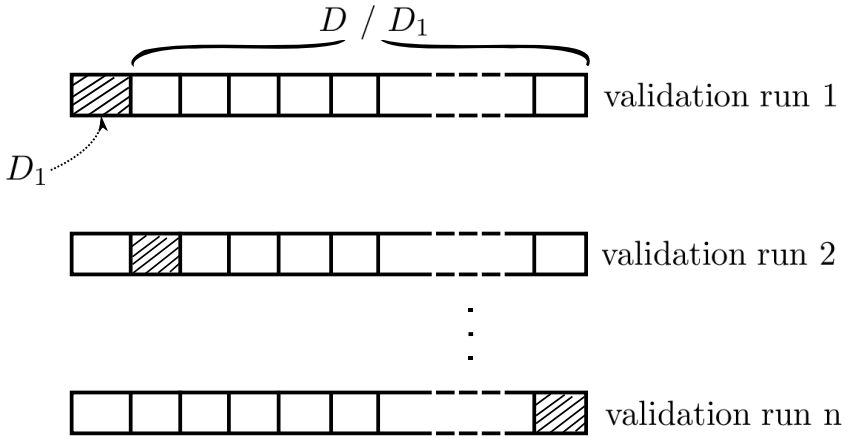
Test set method





Resampling methods: n-fold cross-validation

Observations $D \to n$ disjunct sets D_j, $\bigcup_{j=1}^{n} D_j = D$.



Training of n networks on the training datasets $D \ / \ D_j$.

Estimation of $E^G_{[\underline{\mathbf{w}}]}$:

$$\widehat{E}^G_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{j=1}^{n} \sum_{\alpha \in D_j} e\big(y_T^{(\alpha)}, y\big(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\big)\big)$$

Typical choice: $n \in \{5, \ldots, 10\}$ $\qquad$ $n = p$: leave-one-out cross-validation

The cross-validation estimator provides an unbiased estimate of the expected error of the learning algorithm when applied to same-sized datasets drawn from the same underlying distribution (Bates et al., 2024).

Resampling methods: n-fold cross-validation

Variance of the cross-validation estimator:

$$\mathrm{var}\Big(\widehat{E}^G_{[\underline{\mathbf{w}}]}\Big) = \frac{1}{n-1}\sum_{j=1}^{n}\Big(\underbrace{\widehat{E}^G_{[\underline{\mathbf{w}}]\,j}}_{\substack{\text{test error}\\ \text{on } D_j}} - \Big\langle \widehat{E}^G_{[\underline{\mathbf{w}}]}\Big\rangle\Big)^2$$

The estimator of the variance is biased (Bengio and Grandvalet, 2004).

Remarks

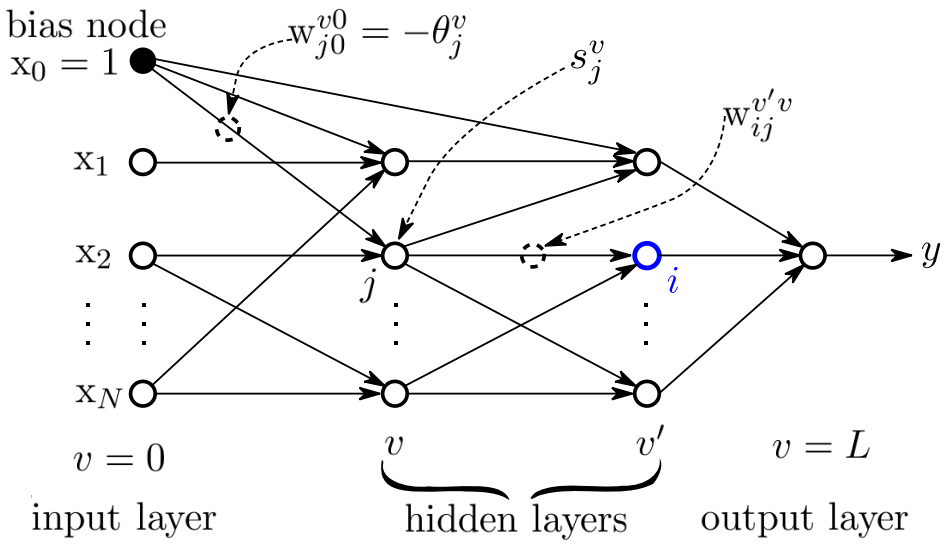
- n-fold cross-validation is only used for estimating E^G.
- Each of the n folds results in a different solution (set of parameters).
- <u>All data</u> are used for finding the parameters for the final predictor.

End of Section 1.3

the following slides contain

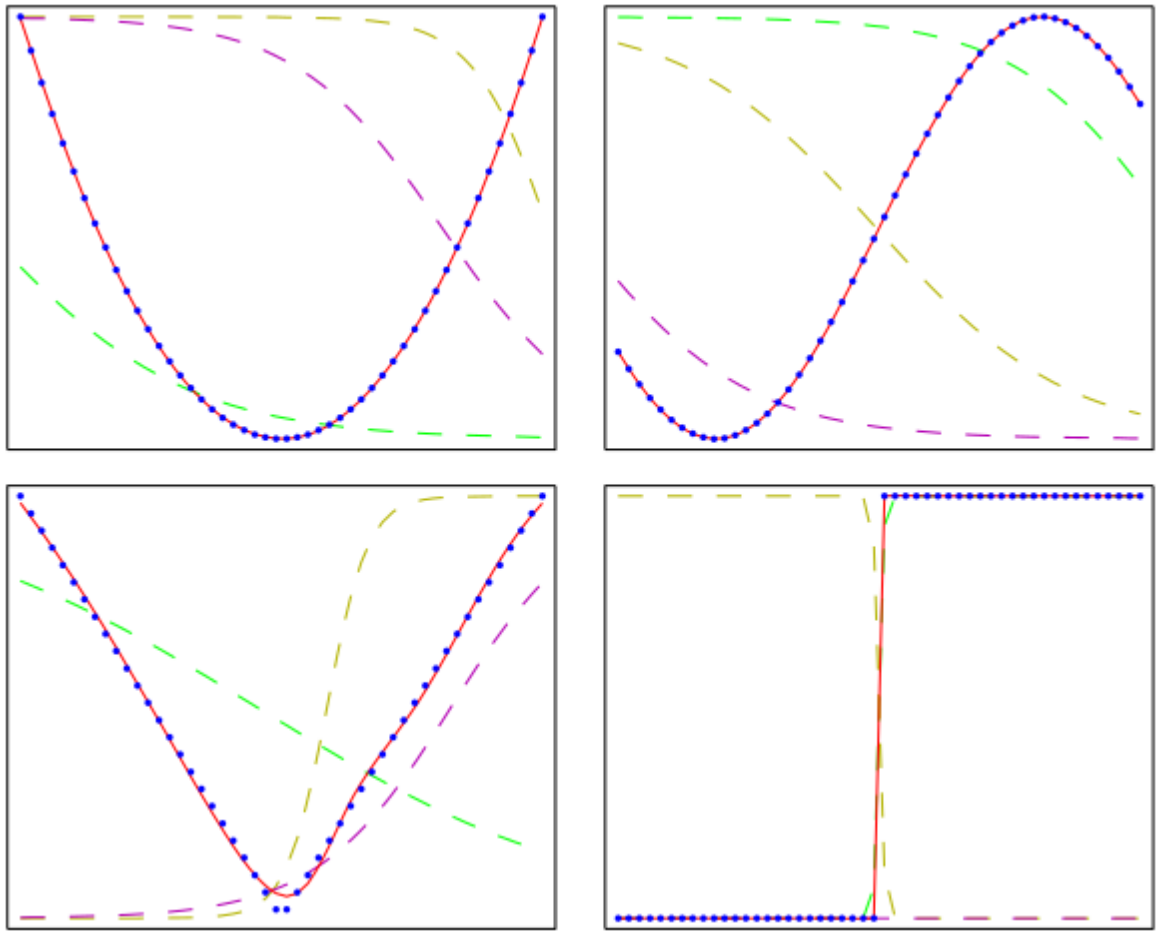
OPTIONAL MATERIAL

Multilayer perceptrons: nomenclature



s_j^v	activity of neuron (v, j) (layer,unit)
$s_0^v = x_0 = 1$	activity of bias neuron
$s_i^0 = x_i$	input to MLP, i^{th} component
$s_i^L = y$	output of MLP
$w_{ij}^{v'v}$	connection weight between neurons (v, j) and (v', i)
$w_{j0}^{v0} = \theta_j^v$	connection weight between bias node and neuron (v, j)
$h_i^{v'}$	$= \sum_j w_{ij}^{v'v} s_j^v$ = total input of neuron (v', i)
$f_i^{v'}$	transfer function of neuron (v', i)

Illustration: ERM with MLPs (Bishop 2009)



fitted MLPs with 1 hidden layer of 3 neurons

References I

David H Ackley, Geoffrey E Hinton, and Terrence J Sejnowski. A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1):147–169, 1985.

S. Bates, T. Hastie, and R. Tibshirani. Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association*, 119(546):1434–1445, 2024.

Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of k-fold cross-validation. *Journal of Machine Learning Research*, 5(1089–1105), 2004.

Ronald E Crochiere and Alan V Oppenheim. Analysis of linear digital networks. *Proceedings of the IEEE*, 63(4):581–595, 1975.

Yann N Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances In Neural Information Processing Systems*, pages 2933–2941, 2014.

Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36:193–202, 1980.

Ken-Ichi Funahashi. On the approximate realization of continuous mappings by neural networks. *Neural Networks*, 2(3):183–192, 1989.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, 2010.

References II

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, 2015. URL https://arxiv.org/abs/1502.01852.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

J J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982. ISSN 0027-8424. doi: 10.1073/pnas.79.8.2554. URL https://www.pnas.org/content/79/8/2554.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.